

Bilkent University, CS 473/573, Session 1, 30pts, 30min

Q1:

- a. Decide whether the following statements are: Always true, Sometimes true, or Never true. Give a brief explanation of your reasoning.
- $f(n) = O(f(n)^2)$
 - $f(n) = \Omega(g(n))$ and $f(n) = o(g(n))$
 - $f(n) = \Theta(f(n/2))$
- b. Give asymptotic upper and lower bounds for $T(n)$ in each of the following recurrences. Assume that $T(n)$ is constant for $n \leq 10$. Make your bounds as tight as possible and justify your answer.
- $T(n) = 64 T(n/4) + 8^{\lg n}$
 - $T(n) = 5 T(n/2) + 10\sqrt{n}$
 - $T(n) = T(n/2) + 2 \log^2 n$

Substitution Method:

- Guess the asymptotic complexity.
- Prove your guess using induction
 - Assume inductive hypothesis holds for $k < n$
 - Try to prove the general case for n

Note: You MUST prove the EXACT inequality, you CANNOT ignore lower order terms. If the proof fails, you could strengthen the inductive hypothesis and try again.

Master Method:

$T(n) = aT(\frac{n}{b}) + f(n)$, $a \geq 1$, $b > 1$, and f is asymptotically positive, then:

- $f(n) = O(n^{\log_b a - \epsilon})$, $\epsilon > 0 \Rightarrow T(n) = \Theta(n^{\log_b a})$
- $f(n) = \Theta(n^{\log_b a} \log^k n)$, $k \geq 0 \Rightarrow T(n) = \Theta(n^{\log_b a} \log^{k+1} n)$
- $f(n) = \Omega(n^{\log_b a + \epsilon})$, $\epsilon > 0$, and $a \cdot f(\frac{n}{b}) \leq c \cdot f(n)$, for some $c < 1 \Rightarrow T(n) = \Theta(f(n))$

You may use the following algorithms and the like covered in the lectures/book as indicated below:

- $INSERTION-SORT(A, n)$: Insertion sort on $A[1 \dots n]$
- $MERGE-SORT(A, p, r)$: Merge sort on $A[p \dots r]$
- $BINARY-SEARCH(A, p, r, key)$: Binary search for key on $A[p \dots r]$
- $H-PARTITION(A, p, r)$: Hoare's partitioning on $A[p \dots r]$, returns the last position of the region $\leq pivot$
- $L-PARTITION(A, p, r)$: Lomuto's partitioning on $A[p \dots r]$, returns the last position of the region $\leq pivot$
- $QUICKSORT(A, p, r)$: Quicksort on $A[p \dots r]$
- $R-QUICKSORT(A, p, r)$: Randomized quicksort on $A[p \dots r]$
- $R-H-PARTITION(A, p, r)$: Randomized Hoare's partitioning on $A[p \dots r]$ with a randomly selected pivot, returns the last position of the region $\leq pivot$
- $R-L-PARTITION(A, p, r)$: Randomized Lomuto's partitioning on $A[p \dots r]$ with a randomly selected pivot, returns the last position of the region $\leq pivot$
- $R-SELECT(A, p, r, i)$: Randomized select algorithm on array $A[p \dots r]$, returns the value of the i th smallest element
- $MM-SELECT(S, n, i)$: Median of medians algorithm on set S containing n elements, returns the value of the i th smallest element
- $HEAP-EXTRACT-MAX(A, n)$: Extract max on the heap of n elements stored in $A[1 \dots n]$
- $MAX-HEAPIFY(A, i, n)$: Max heapify on the i th element of the heap stored in $A[1 \dots n]$
- $BUILD-MAX-HEAP(A, n)$: Builds heap in $A[1 \dots n]$ using the n elements given in $A[1 \dots n]$
- $HEAPSORT(A, n)$: Heapsort on $A[1 \dots n]$
- $MAX-HEAP-INSERT(A, key, n)$: Inserts key into the heap stored in $A[1 \dots n]$
- $HEAP-INCREASE-KEY(A, i, key)$: Increases the value of the i th element of the heap to key
- $COUNTSORT(A, n, k)$: Applies counting sort on $A[1 \dots n]$ of integers in range $[1 \dots k]$